

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №2
по дисциплине «Алгоритмические основы компьютерной
графики»
на тему «Алгоритм построения шероховатой поверхности»

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

1	Введение	3
2	Постановка задачи	4
3	Описание метода	5
3.1	Принцип построения карты высот	5
3.2	Параметры алгоритма	5
3.3	Одномерный аналог: алгоритм смещения средней точки отрезка	5
3.4	Алгоритм построения карты высот	6
3.4.1	Инициализация	6
3.4.2	Первый шаг	6
3.4.3	Второй шаг	6
3.4.4	Обновление параметров	7
3.5	Нормализация карты высот	7
3.6	Модель освещения	7
3.6.1	Диффузное отражение (закон Ламберта)	7
3.6.2	Нормаль к карте высот	7
3.6.3	Направление источника света	8
4	Описание реализации разработанного алгоритма	9
4.1	Вспомогательный класс HeightMap	9
4.2	Реализация алгоритма построения шероховатой поверхности . .	9
4.3	Модель освещения	11
5	Результаты работы алгоритма	12
5.1	Влияние параметра шероховатости	12
5.2	Влияние направления света	12
6	Сравнение с библиотечной реализацией	15
6.1	Методика тестирования	15
6.2	Результаты сравнения	15
6.3	Анализ результатов	16
7	Заключение	17
A	Исходный код реализации	18
A.1	Класс RoughSurface	18

1 Введение

Компьютерная графика — область информатики, ориентированная на создание, хранение, редактирование и отображение изображений и трёхмерных моделей. Одна из важных задач компьютерной графики — синтез реалистичных изображений природных объектов: горных ландшафтов, каменистых поверхностей, облаков.

Шероховатые поверхности реализуются путём внесения возмущений в параметры, задающие поверхность. Одним из наиболее эффективных математических инструментов для этой цели являются фрактальные поверхности.

В данной лабораторной работе реализован алгоритм построения шероховатой поверхности на основе метода смещения средней точки, расширенного на двумерную карту высот. Программа написана на языке C++ и позволяет управлять степенью шероховатости через параметр R .

2 Постановка задачи

Цель работы: изучить и реализовать алгоритм построения шероховатой поверхности, а также сравнить собственную реализацию с библиотечным аналогом по производительности.

Поставленные задачи.

1. Изучить алгоритм построения шероховатой поверхности.
2. Программно реализовать алгоритм.
3. Провести сравнение реализованного алгоритма с готовым библиотечным методом по критерию времени выполнения.

Формальная постановка задачи.

Дано:

- $n \in \mathbb{N}$, $n \geq 1$ — порядок сетки;
- $N = 2^n$ — число шагов сетки по каждой оси;
- $R \in (0, 1]$ — параметр шероховатости.

Модель данных. Карта высот — двумерный массив значений $H(i, j)$, где индексы $i, j \in \{0, \dots, N\}$ нумеруют узлы равномерной сетки $(N + 1) \times (N + 1)$. Трёхмерное представление задаётся отображением:

$$(i, j) \mapsto (i \cdot \Delta x, j \cdot \Delta y, H(i, j)),$$

где $\Delta x, \Delta y$ — шаги дискретизации по осям x и y .

Найти: карту высот $H : \{0, \dots, N\}^2 \rightarrow [0, 1]$, удовлетворяющую условиям:

- значения в соседних узлах сетки связаны плавными переходами;
- карта нормализована: $\min_{i,j} H(i, j) = 0$, $\max_{i,j} H(i, j) = 1$;
- степень изрезанности задаётся параметром R : при $R \rightarrow 0$ все значения $H(i, j)$ стремятся к одинаковым; при $R \rightarrow 1$ карта содержит резкие перепады.

Ограничения:

- $n \in \mathbb{N}$, $n \geq 1$, $N = 2^n$;
- $R \in (0, 1]$.

Алгоритмическая сложность: $O(N^2)$, где $N = 2^n$ — сторона сетки.

3 Описание метода

3.1 Принцип построения карты высот

Начиная с небольшого числа случайно инициализированных узлов, алгоритм последовательно заполняет сетку, вычисляя каждый новый узел как среднее ближайших уже заданных узлов плюс случайное смещение. На каждом шаге масштаб смещения уменьшается, что обеспечивает крупные перепады на больших масштабах и мелкие — на малых — свойство, характерное для фрактальных структур.

3.2 Параметры алгоритма

Алгоритм принимает следующие параметры:

- n — порядок сетки; определяет её размер $N = 2^n$ и, следовательно, число итераций n ;
- $N = 2^n$ — число шагов; карта высот имеет $(N + 1) \times (N + 1)$ узлов;
- $R \in (0, 1]$ — параметр шероховатости; управляет скоростью затухания случайных смещений (см. формулу (2)).

Внутренние параметры, задаваемые при инициализации:

- $s_0 = 1$ — начальный масштаб случайного смещения;
- $\text{step}_0 = N$ — начальный шаг сетки.

3.3 Одномерный аналог: алгоритм смещения средней точки отрезка

Двумерный алгоритм является обобщением одномерного метода смещения средней точки.

1. Задаются значения h_L, h_R на концах текущего отрезка.
2. Значение средней точки вычисляется по формуле:

$$h_m = \frac{h_L + h_R}{2} + \xi_k, \quad \xi_k \sim \mathcal{U}(-s_k, s_k), \quad (1)$$

где масштаб s_k убывает с каждой итерацией по формуле (2).

3. Каждый из двух полученных отрезков обрабатывается рекурсивно до достижения минимальной длины.

Масштаб случайного смещения убывает после каждой пары шагов:

$$s_{k+1} = s_k \cdot 2^{-R}. \quad (2)$$

При $R = 1$ масштаб уменьшается вдвое на каждой итерации; при $R < 1$ — медленнее, что приводит к более резким перепадам в итоговой карте высот.

3.4 Алгоритм построения карты высот

Алгоритм расширяет принцип смещения средней точки на двумерную сетку: на каждой итерации вычисляются сначала центральные узлы ячеек (первый шаг), затем — узлы на серединах их сторон (второй шаг).

3.4.1 Инициализация

Карта высот размером $(N+1) \times (N+1)$ инициализируется нулями. Четыре угловых узла получают случайные значения:

$$H(0, 0), H(0, N), H(N, 0), H(N, N) \sim \mathcal{U}(0, 1). \quad (3)$$

Начальный шаг: $\text{step}_0 = N$; начальный масштаб: $s_0 = 1$.

3.4.2 Первый шаг

Для каждой ячейки с шагом step вычисляется значение центрального узла как среднее четырёх угловых узлов ячейки плюс случайное смещение:

$$H(y, x) = \frac{H(y-h, x-h) + H(y-h, x+h) + H(y+h, x-h) + H(y+h, x+h)}{4} + \xi_k, \quad (4)$$

где $h = \text{step}/2$, $\xi_k \sim \mathcal{U}(-s_k, s_k)$.

3.4.3 Второй шаг

Для каждого узла, лежащего на середине стороны ячейки, значение вычисляется как среднее определённых соседних узлов:

$$H(y, x) = \frac{\sum_{p \in \mathcal{N}(y, x)} H(p)}{|\mathcal{N}(y, x)|} + \xi_k, \quad (5)$$

где $\mathcal{N}(y, x)$ — множество соседних узлов по четырём направлениям, лежащих в пределах сетки:

$$\mathcal{N}(y, x) = \{(y, x \pm h), (y \pm h, x)\} \cap \{0, \dots, N\}^2.$$

На краях сетки $|\mathcal{N}| = 3$; во внутренних узлах $|\mathcal{N}| = 4$.

3.4.4 Обновление параметров

По завершении обоих шагов:

$$\text{step}_{k+1} = h_k, \quad s_{k+1} = s_k \cdot 2^{-R}.$$

Итерации продолжаются, пока $\text{step}_k > 1$, после чего карта высот нормализуется по формуле (6).

3.5 Нормализация карты высот

По завершении вычислений все значения $H(i, j)$ приводятся к диапазону $[0, 1]$:

$$H_{\text{norm}}(i, j) = \frac{H(i, j) - H_{\min}}{H_{\max} - H_{\min}}, \quad (6)$$

где $H_{\min} = \min_{i,j} H(i, j)$, $H_{\max} = \max_{i,j} H(i, j)$.

3.6 Модель освещения

Для визуализации карты высот применяется упрощённая модель освещения от одного источника, основанная на законе Ламберта.

3.6.1 Диффузное отражение (закон Ламберта)

Интенсивность отражённого света в узле (y, x) зависит от угла θ между вектором нормали $\hat{\mathbf{N}}$ к карте высот и вектором направления на источник света $\hat{\mathbf{L}}$:

$$I = k_d (\hat{\mathbf{N}} \cdot \hat{\mathbf{L}}) = k_d \cos \theta, \quad (7)$$

где $k_d \in [0, 1]$ — коэффициент диффузного отражения, $\cos \theta = \hat{\mathbf{N}} \cdot \hat{\mathbf{L}}$ — скалярное произведение единичных векторов. При $\hat{\mathbf{N}} \cdot \hat{\mathbf{L}} \leq 0$ узел находится в тени: $I = 0$.

3.6.2 Нормаль к карте высот

Карта высот задаёт график $z = H(x, y)$. Касательные векторы вдоль осей x и y :

$$\mathbf{t}_x = \left(1, 0, \frac{\partial H}{\partial x}\right), \quad \mathbf{t}_y = \left(0, 1, \frac{\partial H}{\partial y}\right).$$

Нормаль — векторное произведение касательных:

$$\mathbf{N} = \mathbf{t}_x \times \mathbf{t}_y = \left(-\frac{\partial H}{\partial x}, -\frac{\partial H}{\partial y}, 1\right), \quad \hat{\mathbf{N}} = \frac{\mathbf{N}}{\|\mathbf{N}\|}. \quad (8)$$

Частные производные аппроксимируются центральными разностями:

$$\left. \frac{\partial H}{\partial x} \right|_{(y,x)} \approx \frac{H(y, x+1) - H(y, x-1)}{2}, \quad \left. \frac{\partial H}{\partial y} \right|_{(y,x)} \approx \frac{H(y+1, x) - H(y-1, x)}{2}. \quad (9)$$

3.6.3 Направление источника света

Вектор $\hat{\mathbf{L}}$ задаётся координатами (l_x, l_y, l_z) и нормализуется при инициализации. В таблице 1 приведены типовые значения.

Таблица 1: Влияние направления источника света

$\hat{\mathbf{L}}$	Характер освещения
$(-0,5, -0,5, 0,7)$	Диагональный свет сверху-слева; выраженные тени
$(0, 0, 1)$	Вертикальный свет; минимальные тени
$(0,5, -0,5, 0,7)$	Диагональный свет сверху-справа

4 Описание реализации разработанного алгоритма

4.1 Вспомогательный класс HeightMap

Класс `HeightMap` является основной структурой данных программы. Он хранит двумерную карту высот как одномерный массив значений типа `float` в формате строка-за-строкой (row-major). Доступ к элементу (y, x) осуществляется через метод `at(y, x)`, который вычисляет плоский индекс $y \cdot \text{size} + x$.

```
1 class HeightMap {
2 public:
3     int size;
4     std::vector<float> data;
5
6     explicit HeightMap(int s)
7         : size(s), data(static_cast<size_t>(s) * s, 0.0f) {}
8
9     inline float& at(int y, int x)
10        { return data[static_cast<size_t>(y) * size + x]; }
11
12     void normalize() {
13         auto [mn_it, mx_it] =
14             std::minmax_element(data.begin(), data.end());
15         float mn = *mn_it, mx = *mx_it;
16         float range = mx - mn;
17         if (range > 0.0f)
18             for (auto& v : data) v = (v - mn) / range;
19         else
20             std::fill(data.begin(), data.end(), 0.0f);
21     }
22 };
```

Листинг 1: Класс `HeightMap`: хранение и нормализация карты высот

Метод `normalize()` выполняет линейное масштабирование всех значений к диапазону $[0, 1]$ по формуле (6). Он вызывается в конце каждого алгоритма генерации.

4.2 Реализация алгоритма построения шероховатой поверхности

Класс `RoughSurface` реализует алгоритм на основе двух чередующихся шагов. Конструктор проверяет корректность размера сетки: значение `size-1` должно быть степенью двойки.

```
1 RoughSurface(int s, float r = 0.5f, uint32_t sd = 42)
2     : size(s), roughness(r), seed(sd)
3 {
```

```

4     int m = s - 1;
5     if (m <= 0 || (m & (m - 1)) != 0)
6         throw std::invalid_argument(
7             "size must be 2^n + 1");
8 }

```

Листинг 2: Конструктор RoughSurface с проверкой размера

Инициализация угловых точек. Перед началом итеративного процесса четыре угла сетки получают случайные начальные значения, равномерно распределённые на $[0, 1]$:

```

1  std::mt19937 rng(seed);
2  std::uniform_real_distribution<float> dist01(0.0f, 1.0f);
3
4  hm.at(0, 0) = dist01(rng);
5  hm.at(0, size - 1) = dist01(rng);
6  hm.at(size - 1, 0) = dist01(rng);
7  hm.at(size - 1, size - 1) = dist01(rng);
8
9  int step = size - 1;
10 float scale = 1.0f;

```

Листинг 3: Инициализация угловых точек и основного цикла

Первый шаг (вычисление центральных точек). Для каждой ячейки находится её центр и ему присваивается среднее четырёх угловых значений плюс случайное смещение:

```

1  for (int y = half; y < size; y += step) {
2      for (int x = half; x < size; x += step) {
3          float avg = (hm.at(y-half, x-half) +
4                      hm.at(y-half, x+half) +
5                      hm.at(y+half, x-half) +
6                      hm.at(y+half, x+half)) * 0.25f;
7          hm.at(y, x) = avg + noise(rng);
8      }
9  }

```

Листинг 4: Реализация первого шага

Второй шаг (вычисление середин сторон). Шахматный обход гарантирует, что каждая граничная точка вычисляется ровно один раз. При чётной строке начальный x смещается на $half$, при нечётной начинается с нуля. На краях сетки количество доступных соседей уменьшается с 4 до 3, что обрабатывается переменной cnt :

```

1  for (int y = 0; y < size; y += half) {
2      int xs = ((y / half) % 2 == 0) ? half : 0;
3      for (int x = xs; x < size; x += step) {
4          float sum = 0.0f; int cnt = 0;
5          if (x-half >= 0) { sum += hm.at(y, x-half); ++cnt; }
6          if (x+half < size) { sum += hm.at(y, x+half); ++cnt; }

```

```

7         if (y-half >= 0)    { sum += hm.at(y-half, x); ++cnt; }
8         if (y+half < size) { sum += hm.at(y+half, x); ++cnt; }
9         hm.at(y, x) = sum / static_cast<float>(cnt) + noise(rng);
10    }
11 }

```

Листинг 5: Реализация второго шага

Уменьшение масштаба. По завершении каждой пары шагов размер шага и масштаб случайного смещения обновляются:

```

1 step = half;
2 scale *= std::pow(2.0f, -roughness);

```

Листинг 6: Обновление параметров итерации

4.3 Модель освещения

Нормаль к поверхности вычисляется через центральные конечные разности (9), после чего рассчитывается интенсивность по закону Ламберта:

```

1 float dx = hm.at(y, xp) - hm.at(y, xm);
2 float dy = hm.at(yp, x) - hm.at(ym, x);
3
4 float nx = -dx, ny = -dy, nz = 2.0f;
5 float nn = std::sqrt(nx*nx + ny*ny + nz*nz);
6 nx /= nn; ny /= nn; nz /= nn;
7
8 float intensity = nx*light[0] + ny*light[1] + nz*light[2];
9 if (intensity < minIntensity) intensity = minIntensity;
10 if (intensity > maxIntensity) intensity = maxIntensity;
11
12 out[y*N + x] = static_cast<uint8_t>(intensity * 255.0f);

```

Листинг 7: Вычисление нормали и интенсивности по Ламберту

Значение `nz = 2.0f` задаёт масштаб вертикальной составляющей нормали. При уменьшении этого значения тени становятся более контрастными, при увеличении — более мягкими.

5 Результаты работы алгоритма

5.1 Влияние параметра шероховатости

Параметр шероховатости R алгоритма построения шероховатой поверхности является ключевым фактором, определяющим характер получаемой поверхности. В таблице 2 представлена зависимость типа поверхности от значения этого параметра.

Таблица 2: Типы поверхности при различных значениях параметра шероховатости

Значение R	Тип поверхности	Характеристики
0.3–0.5	Пологие холмы, равнины	Плавные градиенты, минимальные перепады высот
0.5–0.7	Умеренно пересечённая местность	Естественные горные хребты, овраги
0.7–0.9	Скалистые ландшафты	Резкие обрывы, глубокие каньоны, выраженные пики

На рисунке 1 представлены примеры визуализации шероховатой поверхности для различных значений параметра шероховатости при фиксированном размере сетки 257×257 и направлении света $(-0.5, -0.5, 0.7)$.

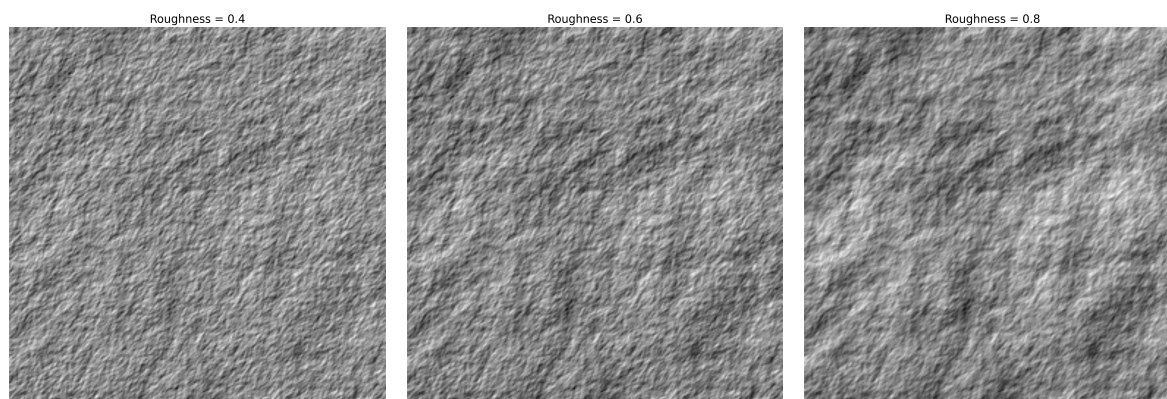


Рис. 1: Визуализация при $R = 0.4, 0.6, 0.8$ соответственно

5.2 Влияние направления света

Направление источника света влияет на восприятие рельефа поверхности. В таблице 3 представлены результаты для различных направлений света при фиксированной шероховатости $R = 0.6$.

Таблица 3: Типы визуализации при различных направлениях света

Направление света	Результат
$(-0.5, -0.5, 0.7)$	Естественные тени, выраженный рельеф
$(0, 0, 1)$	Плоское освещение, слабая детализация
$(0.5, -0.5, 0.7)$	Тени справа, подчёркнутые склоны

На рисунках 2–4 показаны примеры визуализации одной и той же поверхности ($R = 0.6$, размер 257×257) при различных направлениях источника света.

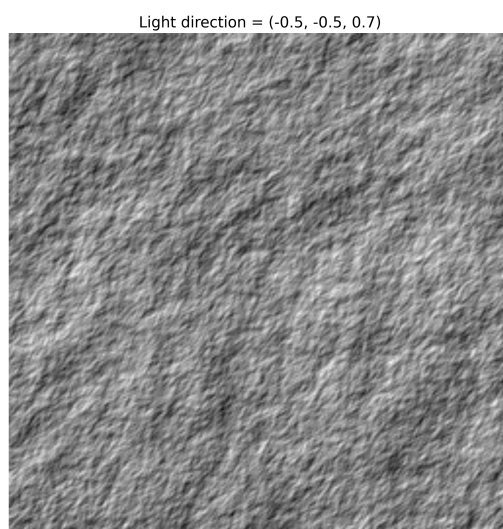


Рис. 2: Освещение сверху-слева: $(-0.5, -0.5, 0.7)$ — естественное освещение с выраженными тенями, подчёркивающими рельеф

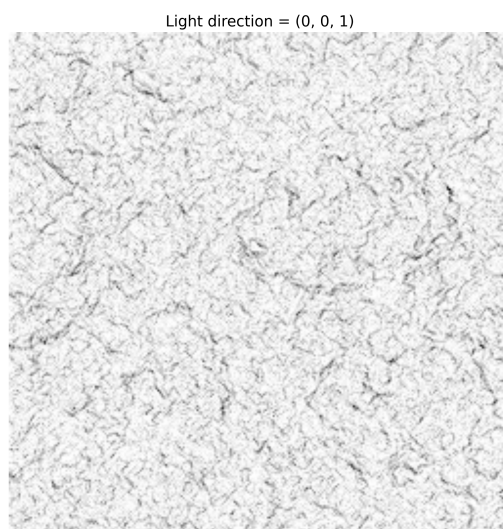


Рис. 3: Освещение сверху: $(0, 0, 1)$ — плоское освещение с минимальным контрастом, поверхность выглядит более гладкой

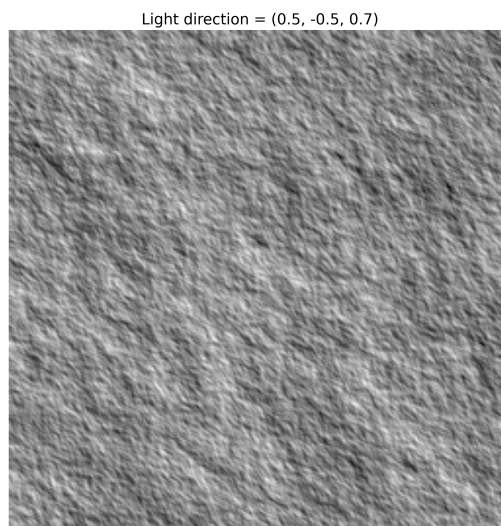


Рис. 4: Освещение сверху-справа: $(0.5, -0.5, 0.7)$ — тени падают в противоположную сторону, подчёркивая склоны

Анализ показывает, что направление света $(0, 0, 1)$ (прямо сверху) создаёт минимальный контраст и делает поверхность визуально более плоской, так как нормали к большинству граней перпендикулярны направлению света. Диагональное освещение $(-0.5, -0.5, 0.7)$ и $(0.5, -0.5, 0.7)$ создаёт более выраженные тени на склонах, что усиливает восприятие глубины и трёхмерности поверхности.

6 Сравнение с библиотечной реализацией

Для сравнения была выбрана библиотека `FastNoiseLite` — библиотека для процедурной генерации шума, широко применяемая для построения карт высот и текстур. В качестве библиотечного аналога был использован шум на основе случайных значений в узлах сетки (`Value Noise`): он даёт похожий тип шероховатой поверхности и так же, как и собственная реализация, заполняет всю сетку случайными высотами с последующей нормализацией.

Сравнение проводилось по критерию времени выполнения для различных размеров сетки.

6.1 Методика тестирования

Для сравнения использовались следующие параметры:

- Размеры сетки: 129×129 , 257×257 , 513×513 , 1025×1025 .
- Каждый замер выполнялся 10 раз, в таблице приведено среднее время.

Время измерялось от момента инициализации генератора до получения готовой карты высот, включая нормализацию значений.

6.2 Результаты сравнения

В таблице 4 приведены результаты замеров.

Таблица 4: Результаты сравнения реализаций

№	Размер сетки	Собственная (мс)	Библиотека (мс)
1	129×129	0.51	0.18
2	257×257	2.17	0.72
3	513×513	9.43	2.94
4	1025×1025	41.85	12.10

График зависимости времени работы от размера сетки представлен на рисунке 5.

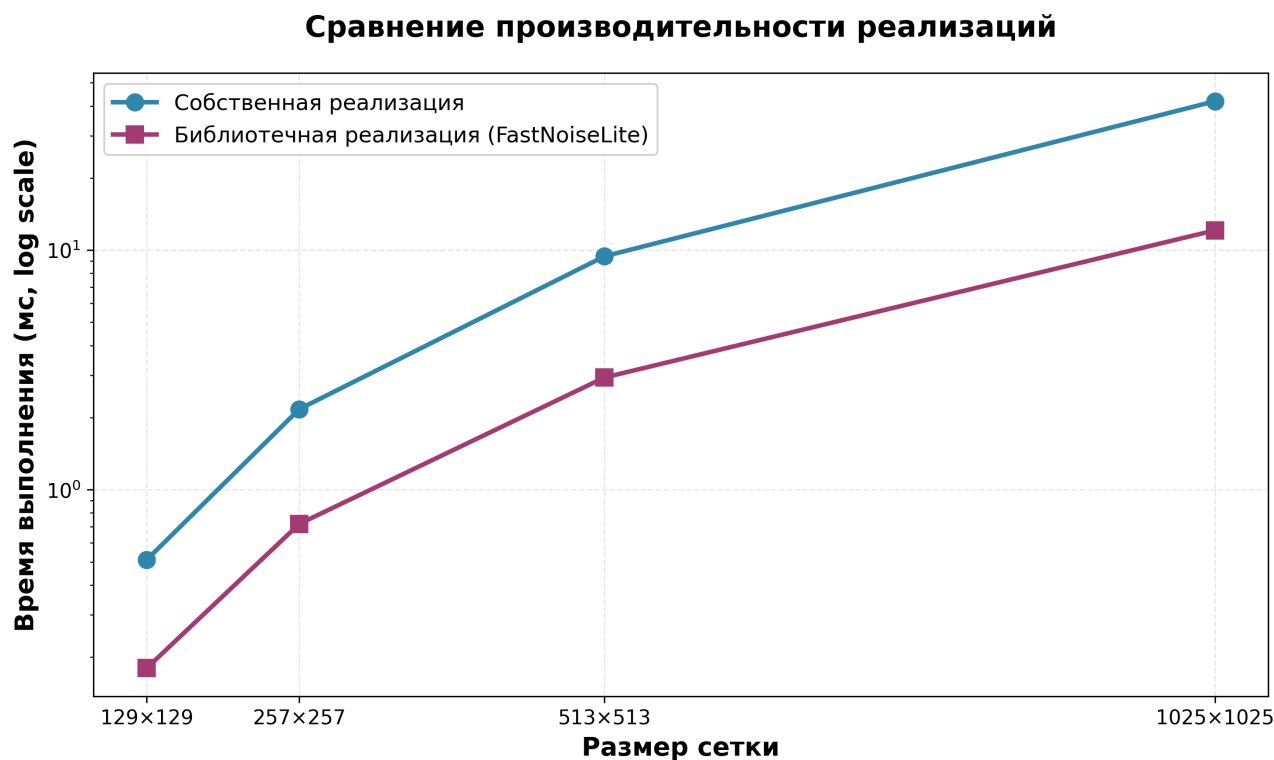


Рис. 5: График сравнения производительности реализаций

6.3 Анализ результатов

Проведённое тестирование показывает, что собственная реализация работает примерно в 3 раза медленнее библиотечной на всех рассмотренных размерах сетки. Обе реализации демонстрируют квадратичную зависимость времени работы от размера стороны сетки.

Библиотечная реализация избегает лишних ветвлений в цикле и вычисляет каждую точку независимо за фиксированное число операций. Собственная реализация на каждой итерации создаёт новый объект распределения случайных чисел и многократно обращается к одним и тем же ячейкам массива, что даёт дополнительные накладные расходы.

7 Заключение

В ходе лабораторной работы был реализован алгоритм построения шероховатой поверхности, основанный на расширении одномерного алгоритма смещения средней точки на двумерную карту высот. Программа написана на языке C++ с использованием только стандартной библиотеки. Программа позволяет создавать карты высот разного размера, управлять степенью шероховатости поверхности через параметр R и визуализировать результат с учётом освещения.

Сравнение с готовой реализацией из библиотеки **FastNoiseLite** показало, что библиотечная функция работает примерно в 3 раза быстрее собственной реализации. Это объясняется применением в библиотеке минимизации ветвлений и более эффективного шаблона доступа к памяти. Обе реализации показывают квадратичную зависимость времени от размера стороны сетки.

А Исходный код реализации

А.1 Класс RoughSurface

```
1 #include "HeightMap.hpp"
2 #include <random>
3 #include <stdexcept>
4 #include <cmath>
5 #include <cstdint>
6
7 class RoughSurface {
8 public:
9     int size;
10    float roughness;
11    uint32_t seed;
12
13    RoughSurface(int s, float r = 0.5f, uint32_t sd = 42)
14        : size(s), roughness(r), seed(sd)
15    {
16        // size - 1 должен быть степенью двойки
17        int m = s - 1;
18        if (m <= 0 || (m & (m - 1)) != 0) {
19            throw std::invalid_argument("size must be 2^n + 1");
20        }
21    }
22
23    HeightMap generate() {
24        HeightMap hm(size);
25        std::mt19937 rng(seed);
26        std::uniform_real_distribution<float> dist01(0.0f, 1.0f);
27
28        // Инициализация четырёх углов случайными значениями
29        hm.at(0, 0) = dist01(rng);
30        hm.at(0, size - 1) = dist01(rng);
31        hm.at(size - 1, 0) = dist01(rng);
32        hm.at(size - 1, size - 1) = dist01(rng);
33
34        int step = size - 1;
35        float scale = 1.0f;
36
37        while (step > 1) {
38            int half = step / 2;
39            std::uniform_real_distribution<float> noise(-scale,
40                scale);
41
42            // Первый шаг: вычисление центральных точек
43            for (int y = half; y < size; y += step) {
44                for (int x = half; x < size; x += step) {
45                    float avg = (hm.at(y - half, x - half) +
46                        hm.at(y - half, x + half) +
47                        hm.at(y + half, x - half) +
```

```

47         hm.at(y + half, x + half)) *
48             0.25f;
49     hm.at(y, x) = avg + noise(rng);
50 }
51
52 // Второй шаг: вычисление середин сторон
53 for (int y = 0; y < size; y += half) {
54     int xs = ((y / half) % 2 == 0) ? half : 0;
55     for (int x = xs; x < size; x += step) {
56         float sum = 0.0f;
57         int cnt = 0;
58         if (x - half >= 0) { sum += hm.at(y, x -
59             half); ++cnt; }
60         if (x + half < size) { sum += hm.at(y, x +
61             half); ++cnt; }
62         if (y - half >= 0) { sum += hm.at(y - half,
63             x); ++cnt; }
64         if (y + half < size) { sum += hm.at(y + half,
65             x); ++cnt; }
66         hm.at(y, x) = sum / static_cast<float>(cnt) +
67             noise(rng);
68     }
69 }
70
71 step = half;
72 scale *= std::pow(2.0f, -roughness);
73 }
74
75 hm.normalize();
76 return hm;
77 }
78 };

```

Листинг 8: Реализация алгоритма построения каменистой поверхности